CASE STUDY 1 – UNIVERSITY SCHOLARSHIP ALLOCATION

1. Problem Statement:

Design a system that evaluates applicants using aggregate marks, subject-wise minimum scores, attendance, and income criteria, ranks eligible students, and resolves ties using predefined rules.

2. Objectives

- Check minimum marks in every subject.
- Calculate aggregate percentage and validate attendance and family income.
- Separate eligible and ineligible applicants.
- Rank eligible applicants using aggregate, Mathematics marks, and alphabetical order.

3. Assumptions

- Three subjects are considered: Mathematics, Physics, and Computer Science; each is out of 100.
- Minimum subject mark is 50; minimum aggregate is 70%.
- Attendance must be at least 75%; annual family income must not exceed ₹2,50,000.
- Ranking: higher aggregate first, then higher Mathematics marks, then alphabetical name.

4. Algorithm

- Store applicants as tuples in a list.
- For each applicant, calculate aggregate and check every eligibility condition.
- Append eligible records with aggregate to one list; append rejected records and reasons to another list.
- Use nested loops to rank eligible records according to the tie-breaking rules.
- Display the ranked list and rejected applicants.

5. Pseudocode

START

Set eligibility thresholds.

Store applicant records as tuples.

FOR each applicant: calculate aggregate and check marks, attendance, and income.

IF all conditions pass, add applicant to eligible list; ELSE add to rejected list.

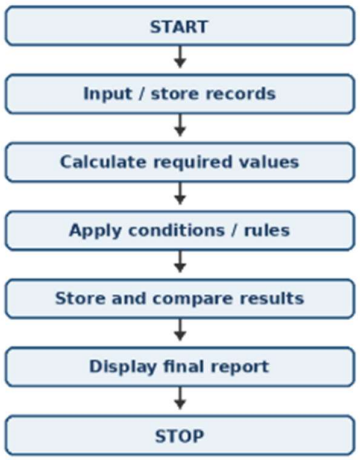
Compare eligible pairs using nested loops and swap when ranking order is wrong.

Display ranking and rejection reasons.

STOP

6. Flowchart

Process flow (high-level):



7. Python Source Code

```
minimum_subject_mark = 50
minimum_aggregate = 70
minimum_attendance = 75
maximum_income = 250000
applicants = [
```

```
    ("Anil", 85, 78, 90, 92, 180000),
    ("Bhavya", 75, 72, 80, 80, 220000),
    ("Charan", 75, 72, 80, 88, 200000),
    ("Divya", 65, 70, 72, 90, 150000),
    ("Esha", 90, 88, 92, 70, 120000)
]
eligible = []
rejected = []
for applicant in applicants:
    name, maths, physics, computer, attendance, income = applicant

    aggregate = (maths + physics + computer) / 3
    reason = ""

    if (maths < minimum_subject_mark or
        physics < minimum_subject_mark or
        computer < minimum_subject_mark):
        reason = "Subject mark below minimum"

    elif aggregate < minimum_aggregate:
        reason = "Aggregate below 70%"

    elif attendance < minimum_attendance:
        reason = "Attendance below 75%"

    elif income > maximum_income:
```

```
        reason = "Income above limit"

    if reason == "":
        eligible.append(
            (name, maths, physics, computer, attendance, income, aggregate)
        )
    else:
        rejected.append((name, round(aggregate, 2), reason))

for i in range(len(eligible)):
    for j in range(i + 1, len(eligible)):
        first = eligible[i]
        second = eligible[j]
        swap = False

        if first[6] < second[6]:
            swap = True

        elif first[6] == second[6] and first[1] < second[1]:
            swap = True

        elif (first[6] == second[6] and
              first[1] == second[1] and
              first[0] > second[0]):
            swap = True
        if swap:
```

```
            eligible[i], eligible[j] = eligible[j], eligible[i]

print("UNIVERSITY SCHOLARSHIP ALLOCATION")
print("--------------------------------")

rank = 1

for student in eligible:
    name, maths, physics, computer, attendance, income, aggregate = student
    print(
        "Rank:", rank,
        "| Name:", name,
        "| Aggregate:", round(aggregate, 2), "%"
    )
    rank += 1
print("\nINELIGIBLE APPLICANTS")
for student in rejected:
    name, aggregate, reason = student
    print(name, "| Aggregate:", aggregate, "% | Reason:", reason)
```

8. Test Cases

Test Case	Applicant	Maths	Physics	Computer Science	Attendance (%)	Annual Income (₹)
1	Anil	85	78	90	92	180000
2	Bhavya	75	72	80	80	220000
3	Charan	75	72	80	88	200000
4	Divya	65	70	72	90	150000
5	Esha	90	88	92	70	120000

9. Sample Output Screenshot:

```
UNIVERSITY SCHOLARSHIP ALLOCATION
--------------------------------
Rank: 1 | Name: Anil | Aggregate: 84.33 %
Rank: 2 | Name: Bhavya | Aggregate: 75.67 %
Rank: 3 | Name: Charan | Aggregate: 75.67 %

INELIGIBLE APPLICANTS
Divya | Aggregate: 69.0 % | Reason: Aggregate below 70%
Esha | Aggregate: 90.0 % | Reason: Attendance below 75%
```

10. Conclusion

The program applies academic, attendance, and income rules, then ranks qualified applicants with deterministic tie-breaking. It demonstrates variables, operators, conditions, loops, lists, tuples, and tuple unpacking.

CASE STUDY 2 – PROGRESSIVE ELECTRICITY BILLING

1. Problem Statement

Calculate household electricity bills using consumption slabs, fixed charges, surcharges, and rebates; compare monthly bills and identify unusual increases.

2. Objectives

- Calculate energy charges using progressive slabs.
- Add fixed charges and a surcharge when applicable.
- Apply a rebate for low consumption.
- Compare monthly bills and flag an unusual increase.

3. Assumptions

- First 100 units cost ₹2.00/unit; next 100 cost ₹3.00/unit; units above 200 cost ₹5.00/unit.
- Fixed charge is ₹50. A 5% surcharge applies when energy charges exceed ₹500.
- A 5% rebate on energy charges applies when consumption is at most 100 units.
- An increase greater than 30% from the previous month is unusual.

4. Algorithm

- Store monthly unit readings in a list.
- Calculate slab-wise energy charges for each month.
- Apply rebate or surcharge, then add fixed charge.
- Compare each bill with the previous month's bill.
- Display each bill and flag increases above 30%.

5. Pseudocode

START

Store monthly consumption.

FOR each month: calculate slab charges.

Apply rebate if units <= 100; otherwise apply surcharge if energy charge > 500.

Add fixed charge and store total bill.

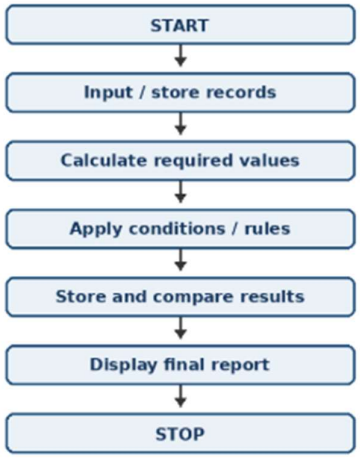
Compare current bill with previous bill and calculate percentage increase.

Display bills and unusual increases.

STOP

6. Flowchart

Process flow (high-level):



7. Python Source Code

```
monthly_units = [80, 120, 190, 280]
fixed_charge = 50
bills = []
for month in range(len(monthly_units)):
  units = monthly_units[month]
  if units <= 100:
    energy_charge = units * 2
  elif units <= 200:
    energy_charge = 100 * 2 + (units - 100) * 3
  else:
    energy_charge = 100 * 2 + 100 * 3 + (units - 200) * 5
```

```
    adjustment = 0
    if units <= 100:
        adjustment = -energy_charge * 0.05
    elif energy_charge > 500:
        adjustment = energy_charge * 0.05
    total_bill = energy_charge + adjustment + fixed_charge
    bills.append(total_bill)
    print(
        "Month", month + 1,
        "| Units:", units,
        "| Energy:", round(energy_charge, 2),
        "| Adjustment:", round(adjustment, 2),
        "| Bill:", round(total_bill, 2)
    )
print("MONTH-TO-MONTH COMPARISON")
for i in range(1, len(bills)):
    increase = bills[i] - bills[i - 1]
    percent = increase / bills[i - 1] * 100
    print("Month", i + 1, "increase:", round(percent, 2), "%")
    if percent > 30:
        print("Unusual increase detected")
```

8. Test Cases:

Test Case	Input / Condition	Expected Result
1	Energy ₹160; 5% rebate ₹8; fixed ₹50	Bill ₹202.00
2	Energy ₹260; no adjustment; fixed ₹50	Bill ₹310.00
3	Energy ₹900; 5% surcharge ₹45; fixed ₹50	Bill ₹995.00

9. Sample Output Screenshot:

```
Month 1 | Units: 80 | Energy: 160 | Adjustment: -8.0 | Bill: 202.0
Month 2 | Units: 120 | Energy: 260 | Adjustment: 0 | Bill: 310
Month 3 | Units: 190 | Energy: 470 | Adjustment: 0 | Bill: 520
Month 4 | Units: 280 | Energy: 900 | Adjustment: 45.0 | Bill: 995.0
MONTH-TO-MONTH COMPARISON
Month 2 increase: 53.47 %
Unusual increase detected
Month 3 increase: 67.74 %
Unusual increase detected
Month 4 increase: 91.35 %
Unusual increase detected
```

10. Conclusion

The program calculates progressive tariffs, adjustments, and fixed charges, then detects large month-to-month changes using lists, arithmetic, conditions, and loops.

CASE STUDY 3 – HOSPITAL BILLING AND PATIENT CLASSIFICATION

1. Problem Statement

Process consultation, diagnostic, and treatment charges, apply conditional concessions, validate billing constraints, and classify patients according to a supplied payment policy.

2. Objectives

- Calculate a patient's total bill from multiple charge categories.
- Validate that entered charges are non-negative.
- Apply concession rules for senior citizens or low-income patients.
- Classify payment status using the final amount.

3. Assumptions

- Consultation, diagnostics, and treatment charges cannot be negative.
- Senior citizens receive a 10% concession on the subtotal.
- Low-income patients receive an additional 5% concession after the senior concession, if applicable.
- Payment status is 'Paid' if amount paid covers the final bill; otherwise 'Balance Due'.

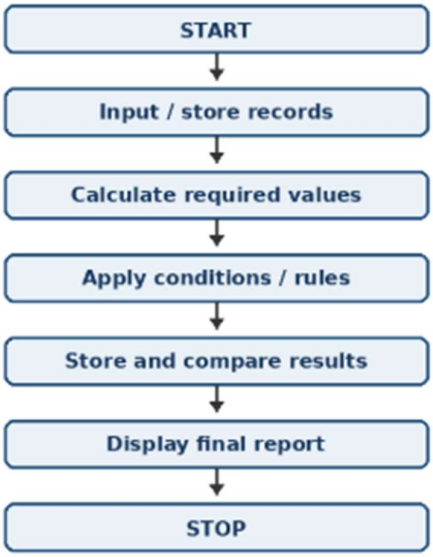
4. Algorithm

- Store patient records as tuples.
- Calculate subtotal for each patient.
- Apply applicable concessions in sequence.
- Calculate final bill and outstanding balance.
- Display bill and payment classification.

5. Pseudocode

- START
- Store patient records.
- FOR each patient: validate all charges and amount paid.
- Calculate subtotal = consultation + diagnostics + treatment.
- Apply senior concession and low-income concession if eligible.
- Calculate final bill and balance.
- Classify as Paid or Balance Due.
- Display patient statement.
- STOP

6. Flowchart



7. Python Source Code

```
patients = [
    ("Ravi", 500, 1200, 3500, "Y", "N", 4500),
    ("Meena", 400, 800, 1800, "N", "Y", 2500),
    ("Kiran", 300, 500, 1000, "N", "N", 1800)
]
for patient in patients:
    name, consultation, diagnostics, treatment, senior, low_income, paid = patient
    if consultation < 0 or diagnostics < 0 or treatment < 0 or paid < 0:
        print(name, "- Invalid billing data")
    else:
        subtotal = consultation + diagnostics + treatment
        concession = 0
        if senior == "Y":
```

```
    concession = subtotal * 0.10
after_senior = subtotal - concession
if low_income == "Y":
    extra_concession = after_senior * 0.05
else:
    extra_concession = 0
final_bill = after_senior - extra_concession
balance = final_bill - paid
if balance <= 0:
    status = "Paid"
    balance = 0
else:
    status = "Balance Due"
print(
    "Patient:", name,
    "| Subtotal:", round(subtotal, 2),
    "| Concession:", round(concession + extra_concession, 2),
    "| Final bill:", round(final_bill, 2),
    "| Paid:", paid,
    "| Balance:", round(balance, 2),
    "| Status:", status
)
```

8. Test Cases:

Test Case	Patient	Input / Condition	Expected Result
1	Ravi	Subtotal ₹5200; Senior citizen concession 10%; Paid ₹4500	Final Bill: ₹4680; Balance: ₹180; Balance Due
2	Meena	Subtotal ₹3000; Low-income concession 5%; Paid ₹2500	Final Bill: ₹2850; Balance: ₹350; Balance Due
3	Kiran	Subtotal ₹1800; No concession; Paid ₹1800	Final Bill: ₹1800; Balance: ₹0; Paid

9. Sample Output Screenshot:

```
Patient: Ravi | Subtotal: 5200 | Concession: 520.0 | Final bill: 4680.0 | Paid: 4500
| Balance: 180.0 | Status: Balance Due
Patient: Meena | Subtotal: 3000 | Concession: 150.0 | Final bill: 2850.0 | Paid:
2500 | Balance: 350.0 | Status: Balance Due
Patient: Kiran | Subtotal: 1800 | Concession: 0 | Final bill: 1800 | Paid: 1800 |
Balance: 0 | Status: Paid
```

10. Conclusion

The solution validates charges, applies policy-based concessions, and determines outstanding balances. It uses tuples, unpacking, conditions, loops, and arithmetic.

CASE STUDY 4 – EMPLOYEE PAYROLL AND DEDUCTION ANALYSIS

1. Problem Statement

Calculate gross salary from basic pay, allowances, overtime, and incentives; apply progressive deductions and generate a formatted salary statement.

2. Objectives

- Calculate allowances, overtime, incentives, and gross salary.
- Apply deductions according to salary bands.
- Calculate net salary.
- Print a readable payroll statement for each employee.

3. Assumptions

- HRA is 20% of basic pay; travel allowance is 8% of basic pay.
- Overtime pay is ₹200 per hour; incentive is supplied per employee.
- Deduction is 5% of gross salary up to ₹25,000, 10% on the portion from ₹25,001 to ₹50,000, and 15% on the portion above ₹50,000.
- All amounts are in Indian rupees.

4. Algorithm

- Store employee details as tuples.
- Calculate HRA, travel allowance, overtime, and gross salary.
- Calculate progressive deduction by salary bands.
- Subtract deduction from gross salary to get net salary.
- Display each salary statement.

5. Pseudocode

START

Store employee payroll records.

FOR each employee: calculate HRA, travel allowance, and overtime pay.

Calculate gross salary.

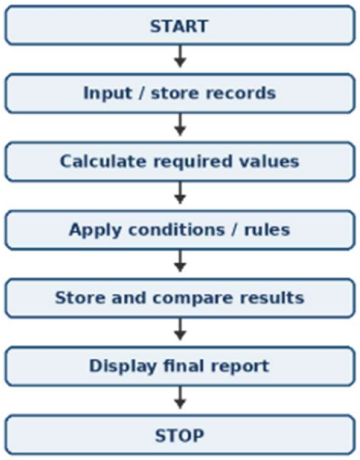
Calculate progressive deduction on each salary band.

Calculate net salary = gross salary - deduction.

Display salary statement.

STOP

6. Flowchart



7. Python Source Code

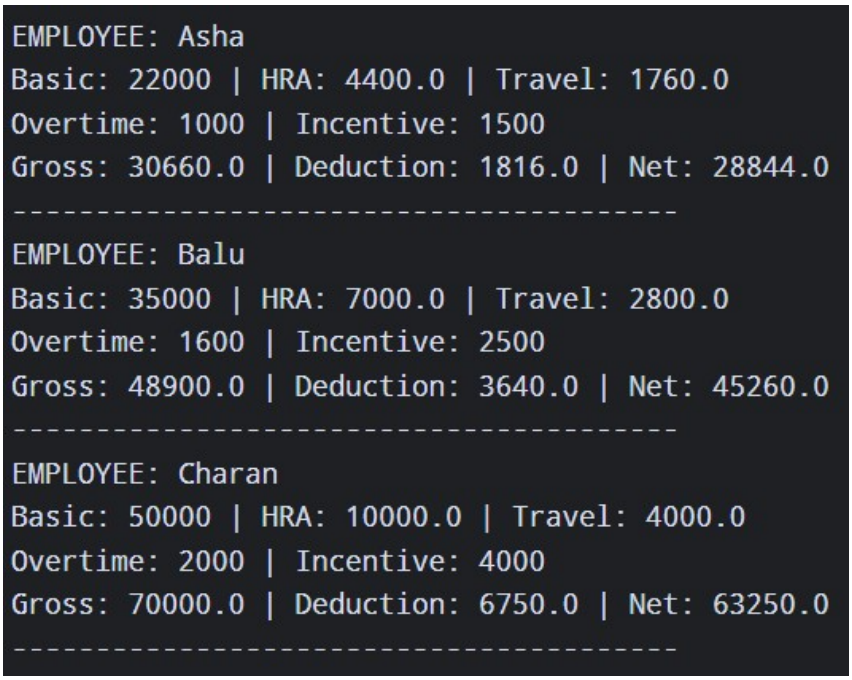
```
employees = [
    ("Asha", 22000, 5, 1500),
    ("Balu", 35000, 8, 2500),
    ("Charan", 50000, 10, 4000)
]
for employee in employees:
    name, basic, overtime_hours, incentive = employee
    hra = basic * 0.20
    travel = basic * 0.08
    overtime = overtime_hours * 200
    gross = basic + hra + travel + overtime + incentive
    if gross <= 25000:
        deduction = gross * 0.05
    elif gross <= 50000:
        deduction = 25000 * 0.05 + (gross - 25000) * 0.10
    else:
```

```
        deduction = 25000 * 0.05 + 25000 * 0.10 + (gross - 50000) * 0.15
    net = gross - deduction
    print("EMPLOYEE:", name)
    print("Basic:", round(basic, 2), "| HRA:", round(hra, 2), "| Travel:",
round(travel, 2))
    print("Overtime:", round(overtime, 2), "| Incentive:", round(incentive, 2))
    print("Gross:", round(gross, 2), "| Deduction:", round(deduction, 2), "| Net:",
round(net, 2))
    print("----------------------------------------")
```

8. Test Cases

Test Case	Input / Condition	Expected Result
1	Basic ₹22,000; 5 overtime hours; incentive ₹1,500	Gross ₹30,660; deduction ₹1,816; net ₹28,844
2	Basic ₹35,000; 8 overtime hours; incentive ₹2,500	Gross ₹48,900; deduction ₹3,640; net ₹45,260
3	Basic ₹50,000; 10 overtime hours; incentive ₹4,000	Gross ₹70,000; deduction ₹6,750; net ₹63,250

9. Sample Output Screenshot



10. Conclusion

The program generates payroll statements and calculates progressive deductions. It demonstrates arithmetic expressions, conditional bands, tuple records, unpacking, and iteration.

CASE STUDY 5 – MULTI-PLAN TELECOMMUNICATIONS COST OPTIMIZATION

1. **Problem Statement**

Design a Python program that compares mobile plans using a user's data usage, call usage, validity, and budget.
Calculate extra charges when usage exceeds a plan's limits and identify the cheapest plan that meets the requirements.

2. **Objectives**

1.Store plan details using tuples and a list.
2. Calculate excess-use charges and total cost for each plan.
3. Check validity and budget constraints.
4. Display the most economical eligible plan

3.Assumptions

1. Each plan is represented as (name, monthly_fee, data_limit_GB, call_limit_minutes, validity_days, extra_data_rate, extra_call_rate).
2. The user enters required data, call minutes, required validity, and budget.
3. Extra charges apply only to usage above the included limit; currency is in rupees**.**

4. Algorithm

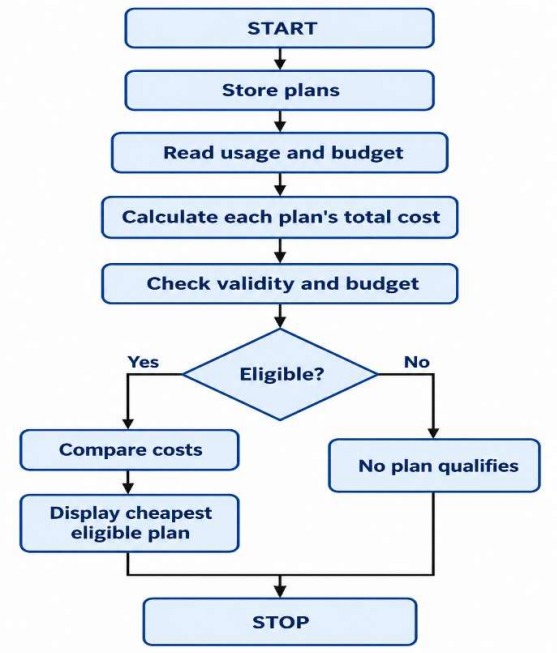
1. Store the mobile plans in a list of tuples.
2. Read the user's required data, call minutes, validity, and budget.
3. For each plan, calculate excess data and excess call minutes.
4. Calculate total cost by adding the monthly fee and excess charges.
5. Keep plans that meet validity and budget requirements.
6. Choose the eligible plan with the lowest total cost; otherwise report that no plan qualifies.

5. Pseudocode

START
Store plan details
Read data usage, call minutes, required validity, and budget

Set best_plan = None
FOR each plan:
Calculate excess data and excess calls
Calculate total cost
IF validity is enough AND total cost <= budget:
IF best_plan is None OR total cost < best_cost:
Update best_plan and best_cost
IF best_plan exists, display it
ELSE display no eligible plan
STOP

6. **Flowchart**



7. **Python Source Code**

```
plans = [
("Basic", 199, 2, 300, 28, 50, 0.50),
```

```
("Smart", 299, 5, 600, 28, 30, 0.40),
("Max", 399, 10, 1000, 56, 20, 0.30)
]

required_data = 4
required_calls = 500
required_validity = 28
budget = 350

best_plan = None
best_cost = 0

for plan in plans:
name, fee, data_limit, call_limit, validity, data_rate, call_rate = plan

extra_data = max(0, required_data - data_limit)
extra_calls = max(0, required_calls - call_limit)

total_cost = fee + extra_data * data_rate + extra_calls * call_rate

if validity >= required_validity and total_cost <= budget:
if best_plan is None or total_cost < best_cost:
best_plan = name
best_cost = total_cost

if best_plan is not None:
```

```
print("Most economical eligible plan:", best_plan)
print("Total cost: Rs.", round(best_cost, 2))
else:
print("No plan meets the given requirements.")
```

8**. Test Cases**

1. ('4 GB, 500 minutes, 28 days, n350', 'Smart plan, n299')
2. ('4 GB, 500 minutes, 28 days, n250', 'No eligible plan')

9**. Sample Output**

Most economical eligible plan: Smart
Total cost: Rs. 299

10. **Conclusion**

The program compares mobile plans, applies extra-use charges, checks budget and validity, and selects the lowest-cost eligible option. It demonstrates tuples, lists, loops, conditionals, and arithmetic operators.

CASE STUDY 6 – RAILWAY FARE AND RESERVATION CALCULATOR

1. **Problem Statement**

Design a Python program to calculate railway fares for multiple passengers using distance-based fares, travel class, age-based concessions, reservation fees, and cancellation penalties.

2. Objectives

1. Calculate base fare using distance and travel class.
2. Apply an age-based concession where eligible.
3. Add reservation fees and calculate cancellation penalties.
4. Display each passenger's fare detail

3. Assumptions

1. Base rates per kilometre are Sleeper = ■1.5, AC = ■3.0, and First Class = ■4.5.

2. Passengers aged 60 or above receive a 20% concession on base fare only.

3. Reservation fee is ■40 per passenger; cancellation penalty is 25% of the fare after concession. These are sample classroom rules, not actual railway tariffs.

4. Algorithm

1. Store passenger details in a list of tuples.
2. For each passenger, select the rate based on travel class.
3. Calculate base fare from distance and rate.
4. Apply the age concession if the passenger is at least 60.
5. Add the reservation fee and calculate the cancellation penalty.
6. Display the fare and cancellation amount.

5. Pseudocode

START

Store passenger records

FOR each passenger: Select rate using travel class

base_fare = distance × rate

IF age >= 60: concession = 20% of base_fare

ELSE: concession = 0

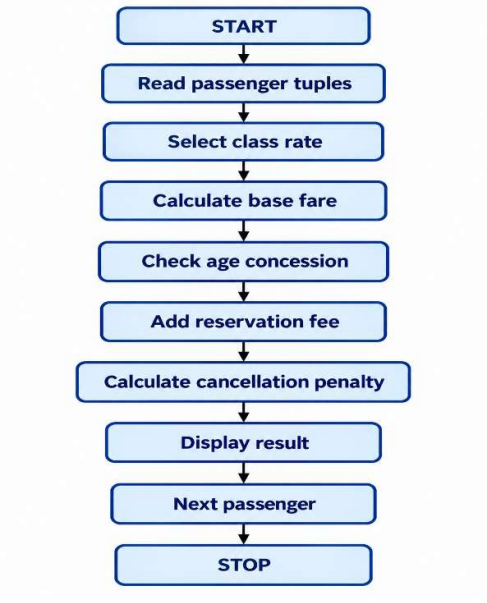
payable = base_fare - concession + reservation_fee

cancellation = 25% of (base_fare - concession)

Display passenger details

STOP

6. Flowchart



7. Python Source Code

```
passengers = [
    ("Anil", 25, 100, "Sleeper"),
    ("Meena", 65, 150, "AC"),
    ("Ravi", 40, 80, "First")
]
rates = {"Sleeper": 1.5, "AC": 3.0, "First": 4.5}
reservation_fee = 40
for passenger in passengers:
    name, age, distance, travel_class = passenger
    base_fare = distance * rates[travel_class]
    if age >= 60:
        concession = base_fare * 0.20
    else:
        concession = 0
    fare_after_concession = base_fare - concession
    total_fare = fare_after_concession + reservation_fee
    cancellation_penalty = fare_after_concession * 0.25
    print("Passenger:", name)
    print("Class:", travel_class)
    print("Total fare: Rs.", round(total_fare, 2))
    print("Cancellation penalty: Rs.", round(cancellation_penalty, 2))
    print("----------------------")
```

8. Test Cases

1. ('Anil, age 25, 100 km, Sleeper', 'Base ■150; total ■190; cancellation ■37.50')

2. ('Meena, age 65, 150 km, AC', 'Base ■450; concession ■90; total ■400; cancellation ■90')

9. Sample Output

Passenger: Anil

Class: Sleeper

Total fare: Rs. 190.0

Cancellation penalty: Rs. 37.5

Passenger: Meena Class: AC

Total fare: Rs. 400.0

Cancellation penalty: Rs. 90.0

Passenger: Ravi

Class: First

Total fare: Rs. 400.0

Cancellation penalty: Rs. 90.

10. Conclusion

The program calculates passenger fares based on distance, class, age concession, and reservation fee, then estimates cancellation penalties. It demonstrates tuples, dictionaries, loops, conditionals, and calculations.

CASE STUDY 7 – RETAIL BILLING WITH MULTIPLE DISCOUNT RULES

1. Problem Statement

Design a Python program to prepare a retail invoice for several products. The program must calculate quantity-based costs, apply a discount when the subtotal reaches a threshold, apply category-specific tax, and limit the discount to a maximum amount.

2. Objectives

1. Store product, price, quantity, and tax category details.
2. Calculate line totals and subtotal.
3. Apply a threshold discount with a maximum limit.
4. Calculate tax and print an itemized invoice.

3. Assumptions

1. Product data is stored as (name, price, quantity, tax_rate).
2. A 10% discount applies when subtotal is at least ■2,000; maximum discount is ■300.
3. Tax is calculated on the discounted subtotal, using a weighted average of item tax rates for this simple example.

4. Algorithm

1. Store product records in a list of tuples.
2. Calculate each line total and the overall subtotal.
3. If subtotal reaches ■2,000, calculate a 10% discount and cap it at ■300.
4. Calculate the weighted tax amount for the items after allocating the discount proportionally.
5. Add tax to the discounted subtotal and display the invoice.

5. Pseudocode

START

Store products Set subtotal = 0 and tax_total = 0

FOR each product: line_total = price × quantity; add to subtotal

IF subtotal >= 2000: discount = minimum of 10% of subtotal and 300

ELSE discount = 0

FOR each product:

Allocate discount proportionally to its line total

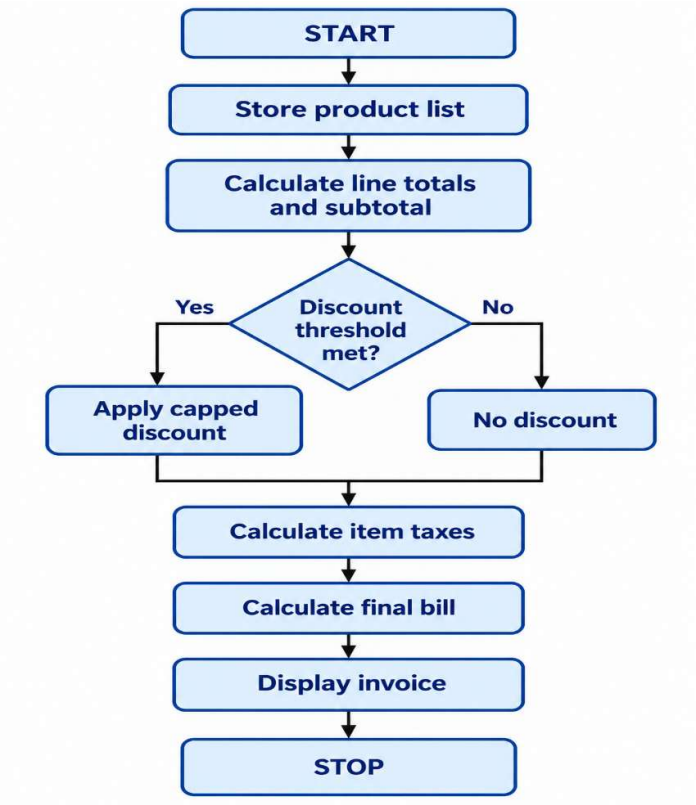
Calculate tax on discounted line total

Calculate final_total = subtotal - discount + tax_total

Display invoice

STOP

6. Flowchart



7. Python Source Code

```
products = [
    ("Notebook", 100, 10, 0.05),
    ("Pen Pack", 50, 10, 0.05),
    ("School Bag", 1200, 1, 0.12)
]

subtotal = 0

# Calculate subtotal
for product in products:
    name, price, quantity, tax_rate = product
    subtotal += price * quantity

# Check discount eligibility
if subtotal >= 2000:
    discount = min(subtotal * 0.10, 300)
else:
    discount = 0

# Calculate tax after discount
tax_total = 0

for product in products:
    name, price, quantity, tax_rate = product

    line_total = price * quantity
    line_discount = discount * line_total / subtotal
    discounted_line = line_total - line_discount
```

```
    tax_total += discounted_line * tax_rate

# Calculate final bill
final_total = subtotal - discount + tax_total

# Display invoice
print("RETAIL INVOICE")
print("------------------------------")

for product in products:
    name, price, quantity, tax_rate = product
    print(name, "x", quantity, "=", round(price * quantity, 2))

print("------------------------------")
print("Subtotal: Rs.", round(subtotal, 2))
print("Discount: Rs.", round(discount, 2))
print("Tax: Rs.", round(tax_total, 2))
print("Final total: Rs.", round(final_total, 2))
```

8. Test Cases

1. ('Subtotal ■2,700', 'Discount ■270; discount is below ■300 cap')
2. ('Subtotal ■4,000', '10% discount is ■400, so capped discount is ■300')

9. Sample Output

```
RETAIL INVOICE
Notebook x 10 = 1000
Pen Pack x 10 = 500
School Bag x 1 = 1200
Subtotal: Rs. 2700
Discount: Rs. 270.0
```

Tax: Rs. 208.05

Final total: Rs. 2638.05

10. Conclusion

The program prepares an itemized invoice, applies a threshold discount with a maximum cap, calculates tax, and prints the final bill. It demonstrates lists, tuples, loops, conditionals, and arithmetic.

CASE STUDY 8 – MANUFACTURING COST AND PROFITABILITY ANALYSIS

1. Problem Statement

Design a Python program to calculate manufacturing cost using material, labour, machine, and wastage costs. Determine the cost per unit, selling revenue, profit, and whether the production meets a required profit target.

2. Objectives

1. Calculate material, labour, machine, and wastage costs.
2. Find total production cost and cost per unit.
3. Calculate revenue and profit.
4. Check whether the profit target is achieved.

3. Assumptions

1. Material cost is ■80 per usable unit; labour is ■25 per unit; machine cost is ■500 per batch.
2. Wastage is 5% of material cost.
3. The batch produces 100 saleable units and each unit sells for ■150.
4. The target profit is ■3,000.

4. Algorithm

1. Read production quantity, unit material cost, labour cost, machine cost, wastage rate, selling price, and target profit.
2. Calculate material and labour costs.
3. Calculate wastage cost as a percentage of material cost.
4. Add all costs to get total production cost.
5. Calculate revenue, profit, and cost per unit.
6. Compare profit with the target and display the result.

5. Pseudocode

START

Set production quantity and cost values

material_cost = quantity × material_rate

labour_cost = quantity × labour_rate

wastage_cost = material_cost × wastage_rate

total_cost = material_cost + labour_cost + machine_cost + wastage_cost

revenue = quantity × selling_price

profit = revenue - total_cost

cost_per_unit = total_cost / quantity

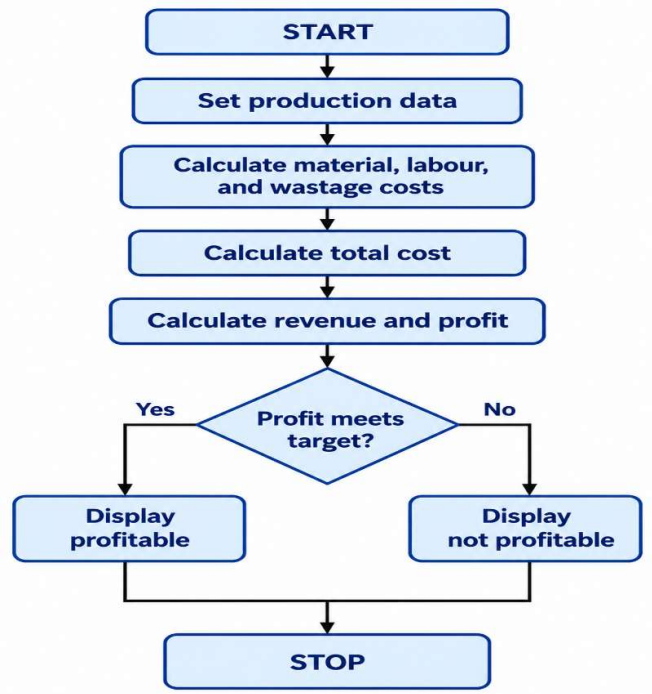
IF profit >= target_profit: print target achieved

ELSE print target not achieved

Display all calculations

STOP

6. Flowchart



7. Python Source Code

```
quantity = 100
material_rate = 80
labour_rate = 25
machine_cost = 500
wastage_rate = 0.05
selling_price = 150
target_profit = 3000
```

```
# Calculate production costs
material_cost = quantity * material_rate
labour_cost = quantity * labour_rate
wastage_cost = material_cost * wastage_rate

total_cost = material_cost + labour_cost + machine_cost + wastage_cost
# Calculate revenue and profit
revenue = quantity * selling_price
profit = revenue - total_cost

# Calculate cost per unit
cost_per_unit = total_cost / quantity

# Display results
print("Material cost: Rs.", material_cost)
print("Labour cost: Rs.", labour_cost)
print("Wastage cost: Rs.", round(wastage_cost, 2))
print("Machine cost: Rs.", machine_cost)
print("Total production cost: Rs.", round(total_cost, 2))
print("Cost per unit: Rs.", round(cost_per_unit, 2))
print("Revenue: Rs.", revenue)
print("Profit: Rs.", round(profit, 2))

# Check profit target
if profit >= target_profit:
    print("Profit target achieved")
else:
    print("Profit target not achieved")
```

8. Test Cases

1. ('100 units at ■150 each', 'Total cost ■11,400; revenue ■15,000; profit ■3,600; target achieved')

2. ('Target profit ■4,000', 'Profit ■3,600; target not achieved')

9. Sample Output

Material cost: Rs. 8000

Labour cost: Rs. 2500

Wastage cost: Rs. 400.0

Machine cost: Rs. 500

Total production cost: Rs. 11400.0

Cost per unit: Rs. 114.0

Revenue: Rs. 15000

Profit: Rs. 3600.0

Profit target achieved

10. Conclusion

The program calculates production costs, wastage, revenue, and profit, then checks the profit target. It demonstrates variables, arithmetic operators, and conditional statements.

CASE STUDY 9 – MULTI-CURRENCY TRAVEL BUDGET ANALYSIS

1. **Problem Statement**
   Design a Python program that converts travel expenses in different currencies into Indian rupees using supplied exchange rates and conversion fees. Compare the total spending with a budget and identify the largest expense category.
2. **Objectives**
   1. Store expenses and exchange rates.
   2. Convert each expense to rupees.
   3. Add conversion fees.
   4. Compare total spending with the budget and identify the largest expense.

3. Assumptions

1. Each expense is represented as (category, amount, currency).

2. Sample exchange rates: INR = ■1, USD = ■83, EUR = ■90.

3. A conversion fee of 2% is applied to USD and EUR expenses; INR expenses have no fee. Rates are illustrative for this exercise.

4. Algorithm

1. Store expense records in a list of tuples and exchange rates in a dictionary.

2. For each expense, convert its amount into rupees.

3. Apply the conversion fee for non-INR currencies.

4. Add each converted expense to the total and track the largest expense.

5. Compare total spending with the budget and display the summary.

5. Pseudocode

START

Store expenses and exchange rates

Set total = 0 and largest_expense = 0

FOR each expense:

Convert amount into INR

IF currency is not INR: add 2% conversion fee

Add converted expense to total

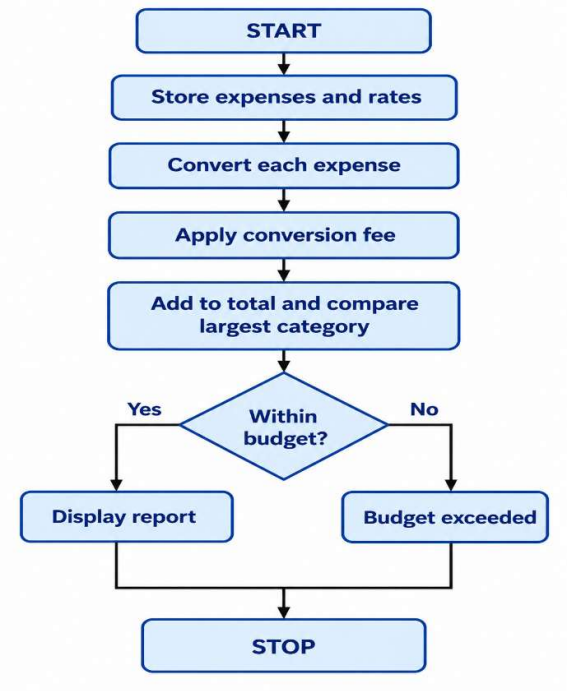
Update largest expense if needed

Compare total with budget

Display total, largest category, and budget status

STOP

6. Flowchart



7. Python Source Code

```
expenses = [
    ("Hotel", 100, "USD"),
    ("Food", 3000, "INR"),
    ("Museum", 30, "EUR"),
    ("Transport", 20, "USD")
]
exchange_rates = {"INR": 1, "USD": 83, "EUR": 90}
budget = 15000
total = 0
largest_category = ""
largest_amount = 0
```

```
for expense in expenses:
    category, amount, currency = expense
    converted = amount * exchange_rates[currency]
    if currency != "INR":
        converted = converted * 1.02
    print(category, ": Rs.", round(converted, 2))
    total = total + converted
    if converted > largest_amount:
        largest_amount = converted
        largest_category = category
print("--------------------------")
print("Total spending: Rs.", round(total, 2))
print("Largest expense category:", largest_category)
if total <= budget:
    print("Within budget")
else:
    print("Budget exceeded by Rs.", round(total - budget, 2))
```

8. Test Cases

1. ('Hotel $100, Food ■3,000, Museum €30, Transport $20', 'Total ■15,913.20; budget ■15,000 exceeded by ■913.20')
2. ('Budget ■16,000', 'Same expenses are within budget')

9. Sample Output

Hotel : Rs. 8466.0

Food : Rs. 3000

Museum : Rs. 2754.0

Transport : Rs. 1693.2

Total spending: Rs. 15913.2

Largest expense category: Hotel

Budget exceeded by Rs. 913.2

10. Conclusion

The program converts expenses into rupees, adds conversion fees, identifies the largest expense, and checks the budget. It demonstrates tuples, dictionaries, loops, conditionals, and arithmetic.

CASE STUDY 10 – CAMPUS EVENT FINANCIAL PLANNING

1. **Problem Statement**
   Design a Python program to calculate campus event expenses, estimate participant contributions, account for sponsorship, identify any financial shortfall, and determine the minimum number of registrations needed to break even

2. Objectives

1. Calculate total event expenses.

2. Subtract sponsorship from total expenses.

3. Calculate the participant fee required for a given registration count.

4. Determine the minimum registrations needed to break even at a fixed fee.

3. Assumptions

1. Fixed costs: venue ■8,000 and equipment ■4,000.

2. Variable costs per participant: catering ■150, transport ■50, and certificate ■20.

3. Sponsorship is ■5,000; expected participants are 100.

4. Registration fee is ■300 per participant.

4. Algorithm

1. Calculate fixed costs and variable cost per participant.

2. Calculate total event cost for expected participants.

3. Subtract sponsorship to find the amount that registrations must cover.

4. Calculate the required contribution per participant.

5. Calculate the minimum registrations needed to break even at the given registration fee, using ceiling division.

6. Display the financial summary.

5. Pseudocode

START

Set fixed costs, variable cost, sponsorship, expected participants, and fee

fixed_cost = venue + equipment

variable_cost = catering + transport + certificate

total_cost = fixed_cost + variable_cost × participants

amount_to_cover = total_cost - sponsorship

required_fee = amount_to_cover / participants

remaining_per_registration = fee - variable_cost

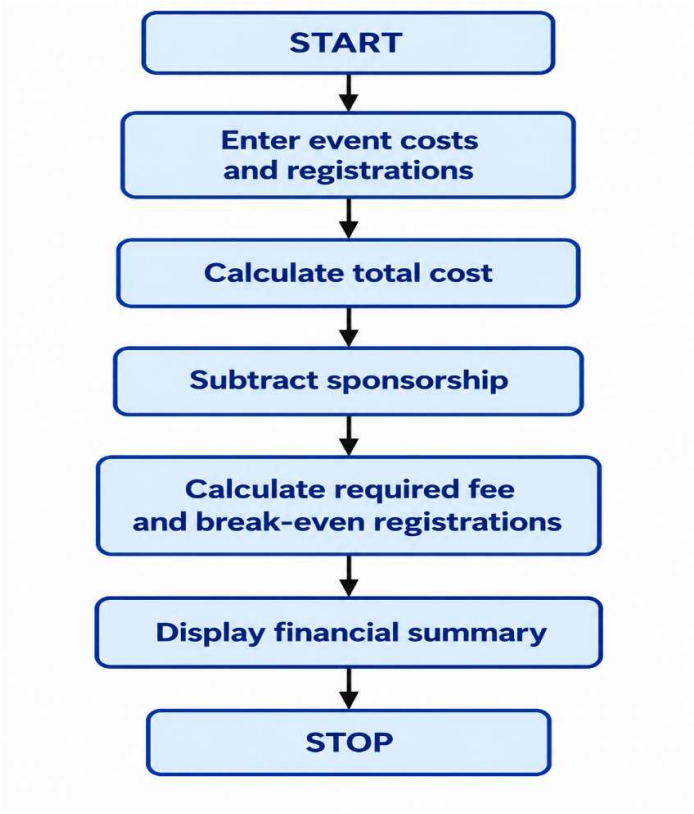
IF remaining_per_registration <= 0: break-even is not possible at this fee

ELSE minimum_registrations = ceiling((fixed_cost - sponsorship) / remaining_per_registration)

Display results

STOP

6. Flowchart



7. Python Source Code

```
venue_cost = 8000
equipment_cost = 4000
catering_per_person = 150
transport_per_person = 50
certificate_per_person = 20
sponsorship = 5000
expected_participants = 100
registration_fee = 300

# Calculate fixed and variable costs
fixed_cost = venue_cost + equipment_cost
variable_cost = catering_per_person + transport_per_person +
certificate_per_person

# Calculate total event cost
total_cost = fixed_cost + variable_cost * expected_participants

# Calculate amount to be covered after sponsorship
amount_to_cover = total_cost - sponsorship
required_fee = amount_to_cover / expected_participants

# Calculate contribution per registration
contribution_per_registration = registration_fee - variable_cost

# Display financial summary
print("Total event cost: Rs.", total_cost)
print("Sponsorship: Rs.", sponsorship)
print("Amount to be covered: Rs.", amount_to_cover)
print("Required fee per participant: Rs.", round(required_fee, 2))

# Calculate break-even registrations
if contribution_per_registration <= 0:
   print("Break-even is not possible at this registration fee.")
else:
   amount_after_sponsorship = fixed_cost - sponsorship
```

```
if amount_after_sponsorship <= 0:
    minimum_registrations = 0
else:
    minimum_registrations = (
        amount_after_sponsorship + contribution_per_registration - 1
    ) // contribution_per_registration

print("Minimum registrations to break even:", minimum_registrations)
```

8. Test Cases

1. ('100 participants, fee ■300', 'Total cost ■34,000; sponsorship ■5,000; required fee ■290 per person; break-even registrations 88')
2. ('Registration fee ■220', 'Contribution after variable cost is ■0; break-even cannot be reached from registrations alone after the fixed-cost gap')

9. Sample Output

Total event cost: Rs. 34000

Sponsorship: Rs. 5000

Amount to be covered: Rs. 29000

Required fee per participant: Rs. 290.0

Minimum registrations to break even: 88

10. Conclusion

The program estimates campus event costs, sponsorship coverage, the fee required per participant, and break-even registrations. It demonstrates variables, arithmetic, conditionals, and integer division.